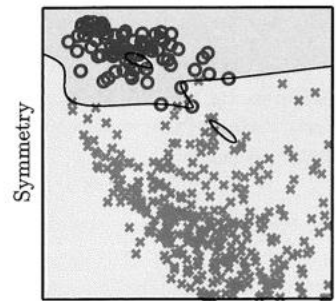
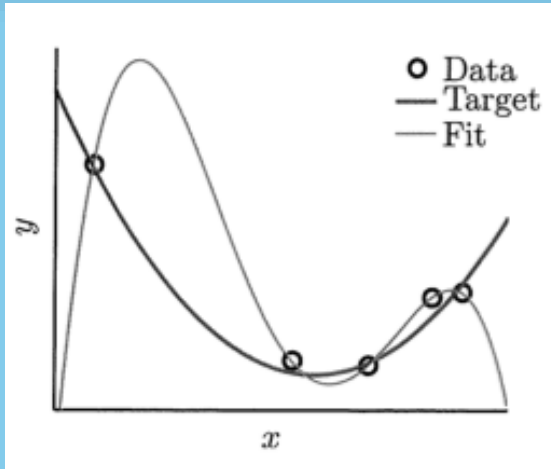




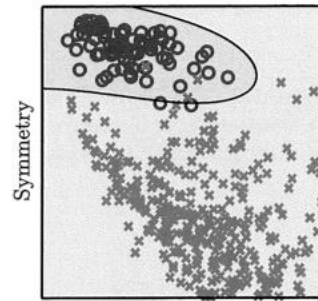
Deep Learning

Dr. Shay Maymon

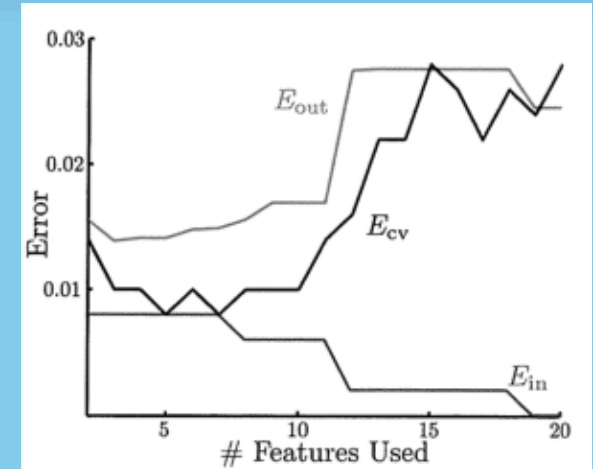
Explain What You See



Average Intensity
20-dim classifier (no validation)
 $E_{in} = 0\%$
 $E_{out} = 2.5\%$



Average Intensity
6-dim classifier (LOO-CV)
 $E_{in} = 0.8\%$
 $E_{out} = 1.5\%$



Regularization

A decorative graphic in the top right corner of the slide, featuring a network of blue nodes connected by lines, resembling a simplified neural network or data flow diagram.

- **Goal in ML:** Perform well on **unseen data**, not just training data
- **Regularization:**

A set of techniques designed to **reduce the test error**
→ **May increase training error** intentionally

Regularization



- There are **many regularization strategies**
- Some introduce **constraints or penalties** to improve generalization:
 - ✓ **Hard Constraints**
Directly restrict model parameters (e.g., limiting weight magnitudes)
 - ✓ **Soft Constraints (Penalties)**
Add extra terms to the objective function
- **Right constraints and penalties → lower test error**

Regularization – Design Goals

- **Design Goals:**
 - ✓ Encode **prior knowledge** (e.g., expected sparsity or smoothness)
 - ✓ Impose a bias toward **simpler models**
 - **Simpler models tend to generalize better**

Limitations of Model Families in Practice



- An overly **complex model** does not **guarantee inclusion** of the true data-generating process
- In most cases, the **true data-generating process** is **unknown** and **inaccessible**
- Deep learning is applied to **highly complex domains** (e.g., images, audio, text) → The **true process** is almost certainly **outside the model family**

Why Regularization Is Essential in Deep Learning

A decorative graphic in the top right corner of the slide, featuring a network of interconnected nodes and lines, resembling a simplified neural network or a data flow diagram.

- Controlling the complexity of the model is not a simple matter of finding the model of the right size, with the right number of parameters
- The best fitting model is a large model that has been regularized appropriately

What is the simplest way to reduce the test error?

- More data = better generalization

But real-world data is often limited

Solution?

Dataset Augmentation



- Create **synthetic ("fake") data** by transforming existing examples
- **Works especially well for classification**
 - Input:** high-dimensional (e.g., image, audio)
 - Output:** single category label
 - Labels** stay valid under many input transformations
- **How it helps:**
 - Generates new (x, y) pairs by modifying x**
 - **Increases diversity** without collecting new samples

Augmenting Images

- **Images are high-dimensional**
Contain many factors of variation (e.g., position, lighting, angle)
- **Many of these variations can be simulated**
→ No need to collect new images

Dataset Augmentation in Object Recognition



Original



Inversion



Scaling



Translation



Rotation



Brightness



Additive Noise

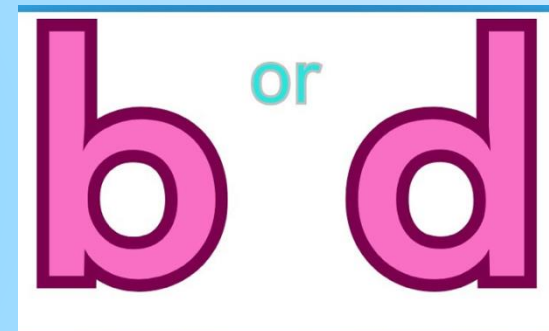
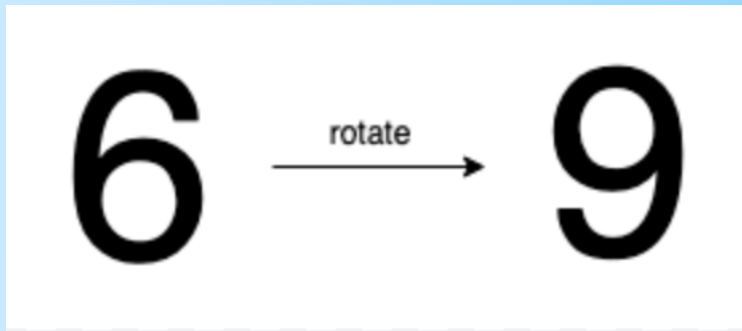


Color Shift

Augmenting Images

- **Common augmentation techniques:**
 - Translation (shift)
 - Rotation
 - Scaling / Zooming
 - Flipping, cropping, noise, color jittering
- These operations maintain the label but increase data diversity
 - Leads to improved generalization

Optical Character Recognition



Dataset Augmentation for Speech Recognition

Goal: Simulate real-world audio variations

→ Improve robustness without collecting new recordings

- **Effective augmentation techniques:**

Noise Addition: Add Gaussian or ambient noise to simulate poor recording conditions

Speed Perturbation: Stretch/compress audio to mimic varying speaking rates

Pitch Perturbation: Shift pitch to reflect speaker or recording differences

Background Mixing: Add background sounds (e.g., traffic, crowd noise)

Benefit: Increases diversity → better generalization



Limitations of Dataset Augmentation

- Hard for **density estimation** tasks
 - ✓ To generate realistic synthetic data, we must already know the data distribution
 - ✓ This defeats the purpose of learning the distribution in the first place

Limitations of Dataset Augmentation

Not all transformations are easy to simulate

Example:

Out-of-plane rotation in images

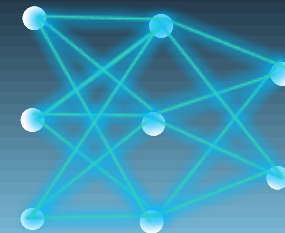
→ Requires 3D modeling, not simple pixel-level manipulation

Limitations of Dataset Augmentation

While **powerful**, augmentation has **domain-specific limitations** and **may not generalize to all learning tasks**

Impact of Hand-Designed Dataset Augmentation

- Carefully chosen **augmentations** can significantly **reduce generalization error**
- **Example:** Small dataset + smart augmentation → performance boost comparable to more data
- Critical for **fair comparisons:**
When comparing algorithms (e.g., A vs. B), ensure they use the same augmentation setup. Otherwise, improvements may come from better augmentation, not a better algorithm



L^2 Regularization - Weight Decay

L^2 Parameter Regularization - Weight Decay

- Adds a **penalty term**: $\|w\|^2$ to the objective function
- Encourages **smaller weights** by pulling them toward the origin
- Helps **prevent overfitting** by smoothing the learned function
- Commonly referred to as:
 - **Ridge Regression** (Statistics)
 - **Tikhonov Regularization** (Applied Mathematics)

L^2 Regularization – Objective and Gradient

- For simplicity, **bias terms are omitted**
- Total objective function:

$$\tilde{J}(\mathbf{w}, X, \mathbf{y}) = \frac{\alpha}{2} \mathbf{w}^T \mathbf{w} + J(\mathbf{w}, X, \mathbf{y})$$

- Gradient with respect to weights:

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}, X, \mathbf{y}) = \alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}, X, \mathbf{y})$$

- Regularization term adds a force **pulling weights toward zero**

L^2 Regularization – Weight Update Rule

- One gradient descent step with weight decay:

$$\mathbf{w} \leftarrow \mathbf{w} - \varepsilon(\alpha \mathbf{w} + \nabla_{\mathbf{w}} J(\mathbf{w}, X, \mathbf{y}))$$

- Equivalently:

$$\mathbf{w} \leftarrow (1 - \varepsilon \alpha) \mathbf{w} - \varepsilon(\nabla_{\mathbf{w}} J(\mathbf{w}, X, \mathbf{y}))$$

- Interpretation:

Shrinkage step: $(1 - \varepsilon \alpha) \mathbf{w}$

Followed by standard gradient update

- This gradual decay helps **prevent weights from growing** too large

L^2 Regularization – Long-term effect

- We've seen how weight decay modifies each update step:

$$\mathbf{w} \leftarrow (1 - \varepsilon\alpha)\mathbf{w} - \varepsilon(\nabla_{\mathbf{w}}J(\mathbf{w}, X, \mathbf{y}))$$

- But what's the **cumulative effect** over training?

Analyzing Regularization: A Quadratic Approximation

- Near a local minimum $\mathbf{w}^*$, the objective $J(\mathbf{w})$ can be approximated by:

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^*)^T H (\mathbf{w} - \mathbf{w}^*)$$

- No first-order (gradient) term
- Hessian H is positive semidefinite

Finding the Minimum of the Objective Function

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^*)^T H(\mathbf{w} - \mathbf{w}^*)$$

The **minimum of $\hat{J}(\mathbf{w})$** occurs where:

$$\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = H(\mathbf{w} - \mathbf{w}^*) = \mathbf{0}$$

Solving the Regularized Quadratic Objective

- We now minimize the regularized approximation

$$\tilde{J}(\mathbf{w}) = \frac{\alpha}{2} \mathbf{w}^T \mathbf{w} + \hat{J}(\mathbf{w})$$

$$\nabla_{\mathbf{w}} \tilde{J}(\tilde{\mathbf{w}}) = \alpha \tilde{\mathbf{w}} + H(\tilde{\mathbf{w}} - \mathbf{w}^*) = \mathbf{0}$$

$$(H + \alpha I) \tilde{\mathbf{w}} = H \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^*$$

- As $\alpha \rightarrow 0$, the unregularized solution $\tilde{\mathbf{w}}$ approaches $\mathbf{w}^*$.

Solving the Regularized Quadratic Objective

- We now minimize the regularized approximation

$$\tilde{J}(\mathbf{w}) = \frac{\alpha}{2} \mathbf{w}^T \mathbf{w} + \hat{J}(\mathbf{w})$$

$$\nabla_{\mathbf{w}} \tilde{J}(\tilde{\mathbf{w}}) = \alpha \tilde{\mathbf{w}} + H(\tilde{\mathbf{w}} - \mathbf{w}^*) = \mathbf{0}$$

$$(H + \alpha I) \tilde{\mathbf{w}} = H \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^*$$

- As $\alpha \rightarrow 0$, the unregularized solution $\tilde{\mathbf{w}}$ approaches $\mathbf{w}^*$

Dataset Augmentation

What happens as α grows?

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^*$$

Dataset Augmentation

What happens as α grows?

$$\tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^*$$

H is **real** and **symmetric**:

$$H = Q \Lambda Q^T$$

$$\tilde{\mathbf{w}} = (Q \Lambda Q^T + \alpha Q Q^T)^{-1} Q \Lambda Q^T \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = (Q(\Lambda + \alpha I)Q^T)^{-1} Q \Lambda Q^T \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = Q(\Lambda + \alpha I)^{-1} \mathbf{Q}^T Q \Lambda Q^T \mathbf{w}^*$$

$$\tilde{\mathbf{w}} = Q(\Lambda + \alpha I)^{-1} \Lambda Q^T \mathbf{w}^*$$

$$(Q^T \tilde{\mathbf{w}}) = (\Lambda + \alpha I)^{-1} \Lambda (Q^T \mathbf{w}^*)$$

Geometric Interpretation of Weight Decay

- Weight decay **rescales** w^* along the axes defined by the eigenvectors of the Hessian H

$$(Q^T \tilde{w}) = (\Lambda + \alpha I)^{-1} \Lambda (Q^T w^*)$$

- For each eigenvector q_i of H with eigenvalue λ_i :

$$(q_i^T \tilde{w}) = \frac{\lambda_i}{\lambda_i + \alpha} (q_i^T w^*) \quad \text{Rescaling factor: } \frac{\lambda_i}{\lambda_i + \alpha}$$

Geometric Interpretation of Weight Decay

- For each eigenvector q_i of H with eigenvalue λ_i :

$$(q_i^T \tilde{w}) = \frac{\lambda_i}{\lambda_i + \alpha} (q_i^T w^*) \quad \text{Rescaling factor: } \frac{\lambda_i}{\lambda_i + \alpha}$$

High-curvature directions ($\lambda_i \gg \alpha$) minimal shrinkage (preserved)

Low-curvature directions ($\lambda_i \ll \alpha$) strong shrinkage (nearly zero)

How Regularization Selectively Shapes the Solution

A decorative graphic in the top right corner consisting of several blue dots connected by thin, light blue lines, forming a network-like structure.

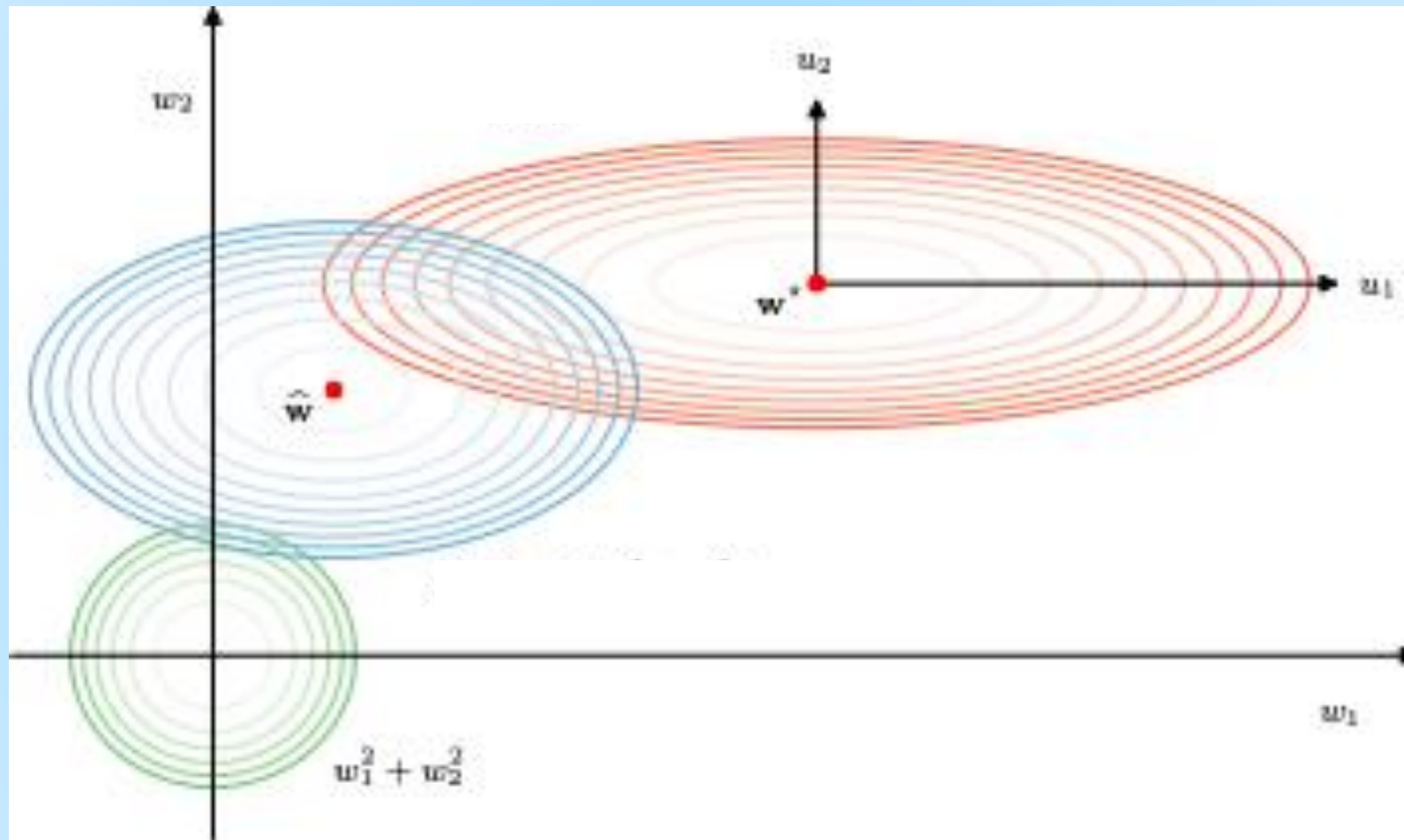
- **Regularization Preserves useful directions**

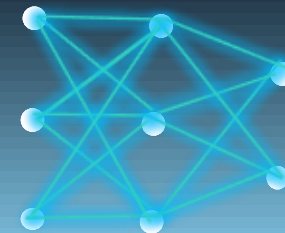
Weight components that help reduce the loss are retained. These align with large Hessian eigenvalues (High curvature), where movement significantly impacts the objective

- **Weight decay Suppresses unimportant directions**

Low-eigenvalue directions (low-curvature) contribute little to loss reduction and are shrunk away during training

How Regularization Selectively Shapes the Solution





L^1 Regularization

L^1 Parameter Regularization

- L^2 weight decay is the **most common** form of weight decay
- L^1 regularization penalizes the **absolute values** of weights:

$$\|\mathbf{w}\|_1 = \sum_i |w_i|$$

- As with L^2 weight decay, L^1 weight decay **controls the strength** of the regularization by scaling the penalty using **a positive hyperparameter** α

L^1 Regularization: Objective and Gradient

- Regularized objective:

$$\tilde{J}(\mathbf{w}, X, \mathbf{y}) = \alpha \|\mathbf{w}\|_1 + J(\mathbf{w}, X, \mathbf{y})$$

- The corresponding **gradient** ($\mathbf{w} \neq \mathbf{0}$)

$$\nabla_{\mathbf{w}} \tilde{J}(\mathbf{w}, X, \mathbf{y}) = \alpha \text{sign}(\mathbf{w}) + \nabla_{\mathbf{w}} J(\mathbf{w}, X, \mathbf{y})$$

- Key **differences** from L^2 :

Gradient is constant ($\pm\alpha$), not proportional to w_i

No closed-form solution from quadratic approximations

Solving L^1 Regularization with a Diagonal Hessian

- Quadratic Approximation:

$$\hat{J}(\mathbf{w}) = J(\mathbf{w}^*) + \frac{1}{2} (\mathbf{w} - \mathbf{w}^*)^T H (\mathbf{w} - \mathbf{w}^*)$$

- Assumption: **Hessian is diagonal**

$$H = \text{diag}([H_{1,1}, \dots, H_{d,d}]), \quad \text{with } H_{i,i} > 0 \forall i = 1, 2, \dots, d$$

- Quadratic approximation of the L^1 regularized objective becomes:

$$J(\mathbf{w}^*) + \sum_i \frac{1}{2} H_{i,i} (w_i - w_i^*)^2 + \alpha |w_i|$$

This setup allows an **element-wise analytical solution**

Analytical Solution for L^1 Regularized Quadratic Approximation with a Diagonal Hessian

- Optimal solution for each parameter w_i :

$$w_i = \text{sign}(w_i^*) \max \left(|w_i^*| - \frac{\alpha}{H_{i,i}}, 0 \right)$$

- Two outcomes based on w_i^* :

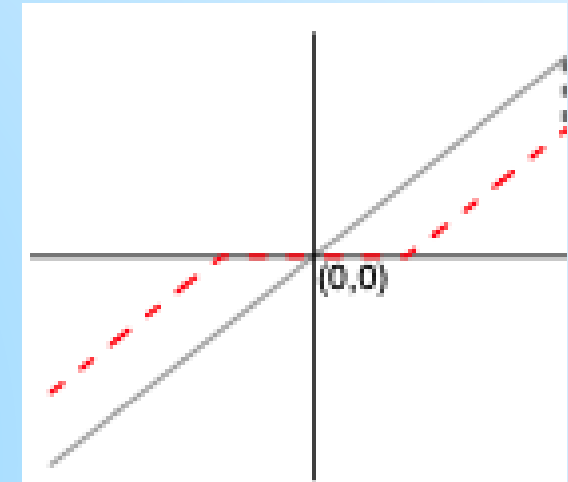
1. Shrink to zero:

$$|w_i^*| < \frac{\alpha}{H_{i,i}} \rightarrow w_i = 0$$

L^1 penalty overpowers the data, setting weight to zero

2. Shift toward zero:

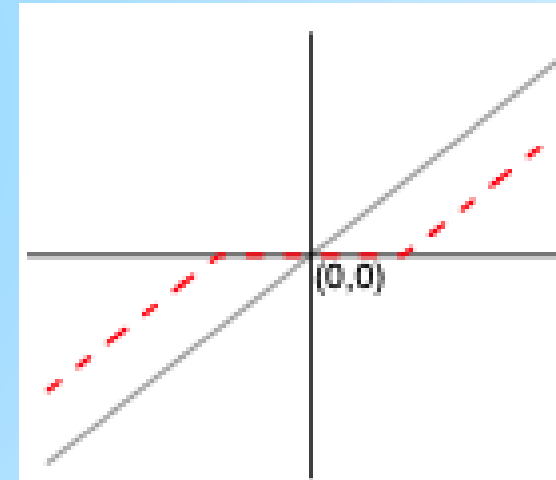
$$|w_i^*| > \frac{\alpha}{H_{i,i}} \rightarrow w_i = \text{sign}(w_i^*) \left(|w_i^*| - \frac{\alpha}{H_{i,i}} \right)$$



Analytical Solution for L^1 Regularized Quadratic Approximation with a Diagonal Hessian

Interpretation:

L^1 regularization introduces a **thresholding effect**, encouraging **sparse models**



L^1 vs. L^2 Regularization: Sparsity Effect

$$L^2: \tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^* \rightarrow \tilde{w}_i = \frac{H_{i,i}}{\alpha + H_{i,i}} w_i^*$$

$$L^1: w_i = \text{sign}(w_i^*) \max\left(|w_i^*| - \frac{\alpha}{H_{i,i}}, 0\right)$$

- L^1 regularization **encourages sparsity**:
Some weights are driven **exactly to zero**, **removing irrelevant features**
- L^2 regularization **shrinks but retains** all weights
Even small remain nonzero
- Key difference:
 - $L^1 \rightarrow$ **Sparse solution** (feature selection), interpretable models
 - $L^2 \rightarrow$ **Dense, Smooth shrinkage** (all features retained, but smaller)

L^1 vs. L^2 Regularization: Sparsity Effect

$$L^2: \tilde{\mathbf{w}} = (H + \alpha I)^{-1} H \mathbf{w}^* \rightarrow \tilde{w}_i = \frac{H_{i,i}}{\alpha + H_{i,i}} w_i^*$$

$$L^1: w_i = \text{sign}(w_i^*) \max\left(|w_i^*| - \frac{\alpha}{H_{i,i}}, 0\right)$$

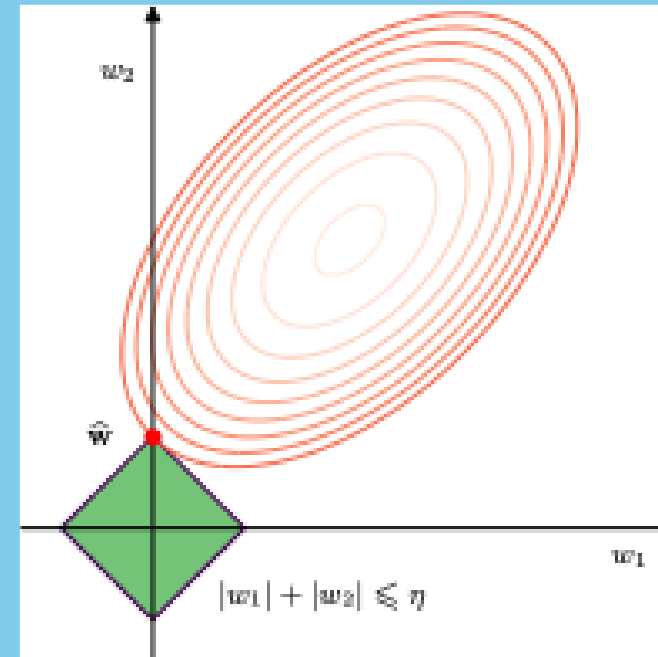
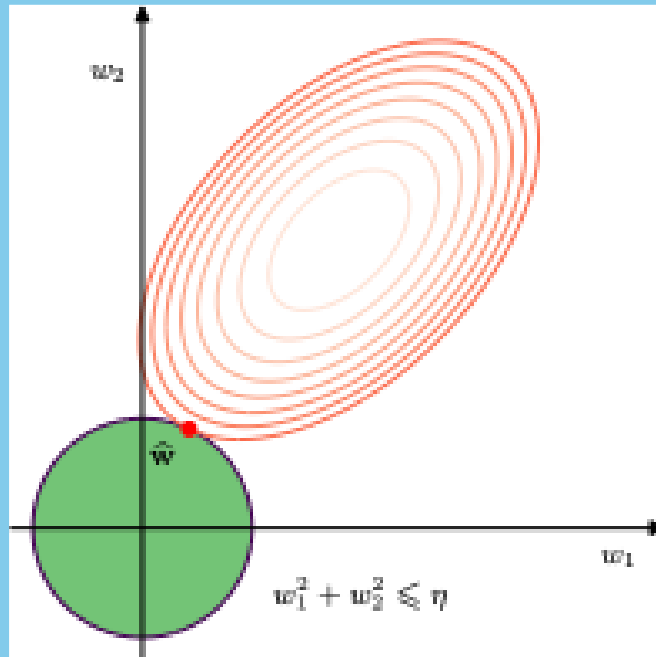
- Sparsity is key for **feature selection**:

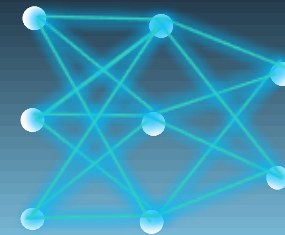
L^1 regularization automatically **selects features** by **eliminating unimportant ones**

- **Simplifies** the learning problem:

Reduces dimensionality, improves interpretability, and can enhance generalization

L^1 vs. L^2 Regularization: Sparsity Effect





Thank you